

Introduction to Generative AI and LLM APIs

Generative AI

Generative AI is a branch of Artificial Intelligence that can generate new content such as text, images, audio, video, and code. Unlike traditional machine learning models that mainly classify or predict predefined outputs, Generative AI creates entirely new outputs based on patterns learned during training.

Examples include:

- Chatbots
- Image generation systems
- Code generation assistants
- Text summarization tools

Generative AI is most suitable for tasks where multiple valid outputs are possible.

Gen ai= used to generate new content

ML models = use to predict , classify and detect by learning patterns from data

Why might predicting sequences (text) be different from predicting numbers?

ML model generates from a small number of possible outputs for example is it going to rain tomorrow?—yes or no

Whereas a gen ai has 1000s of possible options to use as an output while generating a sentence for example

Therefore predicting text is more hard and more open ended than predicting numbers. O/p's are large, context has to be there, sequential outputs

What kinds of problems seem suitable or unsuitable for Generative AI?

Best used when many possible outputs , creativity, language or pattern generation, exact o/p not always required

Not used for precise calculations, guaranteed logic, strict accuracy etc

Large Language Models (LLMs)

LLM stands for Large Language Model.

An LLM is a neural network trained on massive amounts of text data. It learns language patterns and generates text by predicting the next token based on previous tokens.

Examples of LLMs include:

- GPT
- Gemini
- Claude
- Llama

The fundamental objective of an LLM is:

Predict the most probable next token given the previous tokens.

They are basically type of gen ai trained on vast datasets and enormous neural networks to train in and understand human language

Why might the same input sometimes produce different outputs?

Because in gen ai randomness is involved. It samples from several likely words
Different runs may choose different words

Why might an LLM appear confident even when incorrect?

LLMs are trained to generate fluent and plausible text, not to verify facts.
So even if the answer is wrong they generate an answer by understanding the pattern following which an answer is answered

Why might an LLM not behave deterministically even for the same input?

LLMs are often configured to include randomness during generation. When choosing the next token, several options may have similar [probabilities](#) so the model can take first option in first run and take the next most probable one in the second run.

What Happens When Text Is Sent to an LLM

First the sentence is broken into smaller tokens then each token gets converted into a numerical code. Then it is sent to transformer, a neural network. Neural network model analyses the input and predicts the output and a response is generated

Steps are:

1. The text is tokenized.
2. Tokens are converted into numerical representations.
3. The Transformer model processes the tokens.
4. The model predicts the next token.
5. The process repeats until a complete response is generated.

Pipeline:

User Input → Tokens → Transformer → Next Token Prediction → Response

Tokens

A token is the basic unit of text processed by an LLM.

Tokens are not always complete words. A token can be:

- A word
- Part of a word
- A punctuation mark
- A special symbol

Example:

"unbelievable"

may be split into:

"un" + "believ" + "able"

Therefore, the number of tokens and the number of words are not necessarily the same.

Context Window

Every LLM has a context window which specifies the maximum number of tokens it can process at one time.

The context window includes:

- User input
- Previous conversation
- Generated output

If the context window is exceeded:

- Older information may be discarded
- The request may fail
- The model may truncate information

Training vs Using a Model

Training

Training is the process of teaching a model using large datasets.

During training:

- Data is provided
- Predictions are made
- Errors are calculated
- Parameters are adjusted

This process allows the model to learn patterns.

Using a Trained Model

Using a trained model means performing inference.

Inference involves:

- Receiving new input
- Generating predictions
- Returning results

The model does not learn new knowledge during inference.

Why User Input Does Not Update Model Knowledge

When interacting with an LLM, the model performs inference only.

The model:

- Generates responses
- Does not update its parameters
- Does not permanently learn from user prompts

Updating knowledge would require retraining or fine-tuning the model.

APIs

API stands for Application Programming Interface.

An API allows two software systems to communicate with each other.

It acts as an intermediary between:

- Client applications
- Backend services

Example:

Python Program → API → LLM Server

What is an API in general?

Set of rules that allow software applications to communicate with each other
Intermediate between client and llm

What does a request–response cycle look like?

Client—> request—>server—>response—>client

Request–Response Cycle

The request–response cycle consists of:

1. Sending a request.
2. Processing by the server.
3. Receiving a response.

Example:

Prompt:

"What is AI?"

Request sent to API

Response:

"AI stands for Artificial Intelligence."

API Keys

API keys are secret identifiers used to:

- Authenticate users
- Track usage
- Control access
- Enable billing

API keys should never be shared publicly.

A key is usually passed while creating the API client.

Why LLMs Are Exposed Through APIs

LLMs require:

- Large storage
- Powerful GPUs
- Significant computational resources

Instead of distributing the entire model, providers host the model on their servers and provide access through APIs.

Benefits:

- Easier maintenance
- Faster updates

- Better security
- Reduced hardware requirements for users

Environment Variables

Environment variables are operating-system-level settings that store configuration values outside code.

Example:

GEMINI_API_KEY

Advantages:

- Improved security
- No hard-coded secrets
- Easier deployment

Programs can read environment variables during execution.

Implementing an LLM API Program

A simple Python program was created to:

1. Take text input from the user.
2. Send it to an LLM API.
3. Print the generated response.

Example workflow:

User Input

↓

Gemini API

↓

LLM Processing

↓

Response

↓

Print Output

This implementation satisfies the assignment requirements without using frameworks, memory, system prompts, or output optimization.

Factors Affecting LLM Response Speed

Response speed depends on:

- Input token count
- Output token count
- Model size
- Server load
- Network latency
- Available hardware resources

Longer prompts and larger models generally increase response time.